

Clase 140 — CTFs de pwn e ingeniería inversa

Parte: **5 — Explotación de sistemas y binarios** · Fuente: comunidad CTF · docs pwntools · writeups públicos 🕒 Duración estimada: **130 min** · Nivel: **Avanzado**

🎯 Objetivo

Consolidar toda la parte practicando en **CTFs** de categoría *pwn* y *reversing*: entender el formato de las competencias, montar un flujo de trabajo eficiente (triage → hipótesis → primitiva → exploit → flag), gestionar objetivos remotos, y aprender de los writeups. El CTF es el gimnasio donde se afilan las técnicas de explotación e ingeniería inversa de forma legal y motivadora.

⚠️ **Ética:** los retos de CTF son entornos **autorizados** por diseño. No traslades estas técnicas a sistemas de terceros fuera del CTF.

📚 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Organizar** un flujo de trabajo de CTF (categorías, triage, priorización).
- 2. **Resolver** retos de pwn conectando a un servicio remoto con pwntools.
- 3. **Resolver** retos de reversing deduciendo la lógica y la flag.
- 4. **Reutilizar** una plantilla de exploit local/remoto.
- 5. **Aprender** de writeups y documentar sus propias soluciones.

📖 Temas

#	Tema	Por qué importa
1	Formato de CTF (jeopardy/attack-defense)	Saber a qué juegas
2	Categorías pwn/rev y su solapamiento	Enfocar el estudio
3	Triage rápido de un reto	No perder tiempo
4	Plantilla pwntools local/remoto	Velocidad de ejecución
5	Conexión remota y flag	Cerrar el reto
6	Gestión de libc del reto	Fiabilidad del exploit
7	Writeups y aprendizaje	Progresar rápido
8	Plataformas de práctica	Dónde seguir entrenando

El campo de entrenamiento de toda la parte

Los **CTF** (*Capture The Flag*) son competiciones de seguridad donde se resuelven **retos** para obtener **flags** (cadenas secretas que prueban el éxito), y son el mejor entorno para practicar la explotación de binarios: **legal, seguro y con dificultad graduada**. Toda esta parte se consolida resolviendo retos de las categorías **pwn** (explotación de binarios) y **rev** (ingeniería inversa), que se solapan constantemente —para explotar un binario hay que entenderlo primero, y entenderlo es reversarlo—. La clase cierra la parte enseñando a **aplicar** todo lo aprendido bajo la presión y el formato de una competición real, que es también como se entrenan y se contratan los profesionales del sector.

Los formatos y las categorías

Hay dos formatos principales. En **jeopardy** —el más común— hay un tablero de retos por categorías (pwn, rev, web, cripto, forense, OSINT) y puntos según dificultad; se resuelven de forma independiente. En **attack-defense** cada equipo tiene servicios que debe **parchear** (defender) mientras **explota** los de los rivales, un formato más cercano a la operación real. Las categorías de esta parte son **pwn** —explotar un binario, casi siempre remoto, para obtener una shell y leer la flag— y **rev** —entender un binario para deducir la entrada correcta (una contraseña, una clave) que revela la flag—. Se solapan porque un reto de pwn requiere reversar el binario para encontrar la vulnerabilidad, y algunos de rev requieren cierta explotación.



El flujo de un reto de pwn, y la plantilla de pwntools

Resolver un reto de pwn sigue un flujo reproducible que resume la parte. **Triage rápido** (clase 130): `file` y `checksec` sobre el binario para saber arquitectura y mitigaciones, `strings` para pistas, y abrirlo en Ghidra. **Encontrar la vulnerabilidad** leyendo el decompilado —¿hay un `gets`, un `printf` con formato controlado, un `free` sospechoso?—. **Escribir el exploit** con la **plantilla estándar de pwntools**: `pwntools` genera un esqueleto (`pwn template ./bin`) con la estructura ya montada para alternar entre **local** (`process()`) y **remoto** (`remote()`) cambiando un argumento, más `gdb.attach()` para depurar. Se desarrolla y prueba **en local**, y cuando funciona se apunta al **servicio remoto** del reto. Este flujo —triage, encontrar, explotar en local, lanzar en remoto— es el mismo que en un pentest real.

La libc del reto, los writeups y el aprendizaje

Dos detalles prácticos marcan la diferencia. La **gestión de la libc**: los retos de pwn remotos suelen **proporcionar la libc concreta** del servidor (porque los offsets de `system`, `/bin/sh`, los gadgets dependen de la versión, clase 138), y hay que **usar esa libc, no la del sistema propio**, tanto para calcular offsets como para probar en local con el entorno correcto (`pwntools` y herramientas como `pwninit` automatizan enganchar la libc del reto al binario). Olvidar esto es la causa número uno de un exploit que funciona en local y falla en remoto. Y el hábito que de verdad hace crecer: los **writeups** —escribir la solución de un reto tras resolverlo (o leer los de otros para los que no)— es la forma más eficaz de consolidar el aprendizaje, porque obliga a articular *por qué* funcionó el exploit. Las **plataformas de práctica** (pwn.college, ROP Emporium, picoCTF, HackTheBox, y las plataformas de CTFs en vivo) ofrecen retos graduados para todos los niveles. La lección final de la parte es que la explotación de binarios se

aprende **haciendo**: leer sobre buffer overflows no enseña a explotarlos; resolver cien retos, sí. Y el CTF es el entorno donde hacerlo de forma legal, segura y con una comunidad que comparte el conocimiento —el cierre natural de la parte más técnica del programa—.

Definiciones y características

- **CTF (Capture The Flag)**: competencia de seguridad con retos que ocultan una "flag". *Clave*: pwn y rev son las categorías de esta parte.
- **Jeopardy vs Attack-Defense**: por retos independientes vs. defender/atacar servicios en vivo. *Clave*: el formato cambia la estrategia y el ritmo.
- **Triage de reto**: primeras acciones (`file` , `checksec` , `strings` , abrir en Ghidra) para orientar. *Clave*: decide técnica y esfuerzo.
- **Flag**: cadena objetivo, típicamente `flag{ ... }` o `ctf{ ... }` . *Clave*: se obtiene ejecutando código, leyendo memoria o deduciendo lógica.
- **Plantilla de exploit**: script pwntools reutilizable con `args.REMOTE` . *Clave*: pasar de local a remoto sin reescribir.
- **Writeup**: solución documentada de un reto. *Clave*: la mejor fuente de aprendizaje tras intentarlo.

Glosario

Término	Definición concisa
CTF	Competición de seguridad basada en retos y flags
Flag	Cadena secreta que prueba haber resuelto un reto
Jeopardy	Formato de tablero de retos por categorías
Attack-defense	Formato de parchear los servicios propios y atacar los ajenos
pwn	Categoría de explotación de binarios
rev	Categoría de ingeniería inversa
Triage del reto	<code>file</code> , <code>checksec</code> , <code>strings</code> , abrir en Ghidra
Plantilla pwntools	Esqueleto de exploit con local/remoto conmutables
<code>process()</code> / <code>remote()</code>	Ejecución local / conexión al servicio del reto
Libc del reto	La libc del servidor, que hay que usar para los offsets
pwninit	Herramienta que engancha la libc del reto al binario
Writeup	Documentación de la solución de un reto
Plataforma de práctica	<code>pwn.college</code> , ROP Emporium, picoCTF, HackTheBox
Aprender haciendo	La explotación se domina resolviendo retos

Herramientas y preparación

```
pip install pwntools ROPgadget
# Ghidra/r2 para reversing (clases 131-132), gdb+pwndbg para pwn
```

Plataformas legales de práctica: **pwn.college**, **picoCTF**, **HackTheBox**, **pwnable.kr**, **ROP Emporium**, **crackmes.one**.

Laboratorio guiado

Entorno autorizado: retos de CTF/plataformas de práctica.

1. **Triage** de un reto de pwn descargado:

```
bash file chall && checksec ./chall && strings chall | head # abre en Ghidra para localizar la función vulnerable
```

2. Reproduce el bug localmente y determina la primitiva (overflow, fmt, UAF...).

3. Usa una **plantilla** pwntools que sirva local y remoto:

```
python from pwn import * exe = context.binary = ELF("./chall") def conn(): return remote("chall.ctf.io", 1337) if args.REMOTE else process(exe.path) io = conn() # ... construir payload según la primitiva ... io.sendline(payload) io.interactive() # o io.recvall() para leer la flag
```

4. Desarrolla el exploit local hasta obtener shell/flag; luego lánzalo remoto con `python3 exploit.py REMOTE` usando la **misma libc** que provee el reto.
5. **Reversing**: para un reto rev, abre el binario en Ghidra, deduce el algoritmo de validación y recupera la flag (a veces reimplementando la transformación inversa en Python).
6. Escribe tu propio **writeup**: enunciado, análisis, primitiva, exploit y flag. Compáralo con writeups públicos tras resolverlo.

Ejercicios

1. Resuelve un reto de picoCTF de la categoría Binary Exploitation y documenta el flujo.
2. Resuelve un crackme de crackmes.one y publica (para ti) el razonamiento.
3. Adapta tu plantilla pwntools para gestionar un prompt con `sendlineafter`.
4. Identifica la libc de un reto remoto por un leak y ajústala.
5. Completa dos retos de ROP Emporium (ret2win, split).
6. Lee un writeup avanzado y resume una técnica nueva que aprendiste.

Reto verificable

Resuelve de principio a fin un reto de pwn con objetivo remoto (de una plataforma de práctica), obteniendo la flag, y entrega tu exploit + writeup.

Criterio de aceptación: tu `exploit.py REMOTE` recupera la flag del servidor y tu writeup explica el bug, la primitiva y la cadena de explotación de forma reproducible.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Funciona local, no remoto	libc distinta; usa la del reto y recalcula offsets
Te atascas horas en un reto	Vuelve al triage; quizá la categoría/técnica es otra
Flag no aparece	Puede estar en un fichero; ejecuta <code>cat flag.txt</code> tras la shell
Timing/EOF con el servidor	Sincroniza con <code>recvuntil</code> / <code>sendlineafter</code>
Copias exploits sin entender	No aprendes; resuelve primero, luego contrasta writeups

Preguntas frecuentes

 **¿Por dónde empiezo?** picoCTF y pwn.college son ideales para principiantes; ROP Emporium para ROP; pwnable.kr para variedad.

 **¿Está bien leer writeups?** Sí, después de intentarlo. Son la forma más rápida de aprender técnicas nuevas.

 **¿Los CTF sirven para el trabajo real?** Mucho: entrenan las mismas primitivas y mentalidad del research de vulnerabilidades y el pentesting avanzado.

Referencias

- pwn.college — <https://pwn.college/>
- picoCTF — <https://picoctf.org/>
- ROP Emporium — <https://ropemporium.com/>
- CTFtime (calendario y writeups) — <https://ctftime.org/>

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Clase anterior

[Clase 139 — Kernel exploitation: introducción](#)

Siguiente clase

[Clase 141 — Introducción al malware: tipos y taxonomía](#)